



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 5

Team No.: 19

Project Title: Linux Service Process Supervisor

Team Members

Roll Number	Student Name	Signature
2520030380	Yamini	
2520030364	Lekhana	
2520030613	Anikha	

1. Abstract

Linux Service Process Supervisor is a Linux-based Operating Systems project designed to manage, control, and monitor a service process. The project implements a lightweight supervisor program that provides service lifecycle operations such as Start Service, Stop Service, Service Status, Restart Service, and Exit.

When the service is started, the supervisor uses the `fork()` system call to create a child process, and `exec()` is used to execute the test service program. The supervisor maintains the service PID so that it can monitor and control the running process.

The project uses signals for process control. The `SIGTERM` signal is used to safely terminate the service, while `waitpid()` is used to wait for the service process to finish. The restart functionality stops the existing service and starts it again with a new PID.

A process memory monitoring program is also implemented using the Linux `/proc/<PID>/status` interface. It displays information such as process name, state, PID, PPID, `VmSize`, and `VmRSS`. Service execution activity is recorded in a log file.

Overall, the project demonstrates Linux process creation, process execution, process control, signal handling, service lifecycle management, process monitoring, memory observation, logging, and safe supervisor shutdown.

2. Problem Statement

In a Linux operating system, service processes may need to be started, monitored, stopped, or restarted. Managing these processes manually can make it difficult to track their current state and process IDs.

A service may also terminate unexpectedly or need to be stopped safely. Without a simple management mechanism, the user has to manually identify the process and terminate it using separate Linux commands.

This project addresses this problem by developing a Linux Service Process Supervisor. The supervisor provides a simple menu-based interface through which the user can start a service, check its status, stop it, restart it, and safely exit the supervisor.

The project also provides process memory information by reading the Linux `/proc/<PID>/status` file. This helps observe the resource information of the running service.

The system therefore provides a practical demonstration of Linux process lifecycle management using system calls, signals, process IDs, and the `/proc` filesystem.

3. Objectives

1. To understand Linux process creation and process lifecycle management.
2. To create a service process using `fork()`.
3. To execute a service program using `exec()`.
4. To track the service using its Process ID (PID).
5. To check whether the service is currently running or stopped.
6. To terminate the service safely using the `SIGTERM` signal.
7. To use `waitpid()` for safe process termination and synchronization.
8. To implement service restart functionality and observe the new PID after restarting.
9. To monitor process information using the Linux `/proc/<PID>/status` file.
10. To observe process memory values such as `VmSize` and `VmRSS`.
11. To maintain service execution logs.
12. To safely terminate the service before shutting down the supervisor.

4. Proposed Methodology

The project is implemented as a Linux-based C application consisting of a supervisor, a test service, and a process memory monitoring program.

1. **Service Creation:** The supervisor creates the service process using the `fork()` system call. The child process is created from the supervisor process.
2. **Program Execution:** After creating the child process, `exec()` is used to execute the `test_service` program. The supervisor stores the PID of the service.
3. **Service Status:** The supervisor checks whether the service process is active. If the process is running, the supervisor displays its status as **RUNNING** along with its PID.

4. **Service Termination:** When the Stop Service option is selected, the supervisor sends a SIGTERM signal to the service process. The supervisor then waits for the process to terminate successfully.
5. **Service Restart:** The Restart Service option first terminates the existing service and then starts a new service process. The new process receives a different PID, demonstrating the service restart operation.
6. **Process Monitoring:** A separate process_monitor program reads information from /proc/<PID>/status. It displays the process name, state, PID, PPID, VmSize, and VmRSS.
7. **Logging:** Service execution information is maintained in the logs/test_service.log file. This provides a record of service execution and termination events.
8. **Safe Shutdown:** When the Exit option is selected, the supervisor checks whether a service is running. If necessary, it sends SIGTERM to the service and waits for termination before safely shutting down the supervisor.
9. **Testing:** The system was tested by starting the service, checking its status, stopping it, checking the stopped status, restarting it, checking the new service PID, monitoring process memory, and safely shutting down the supervisor.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API	Purpose in the Project
fork()	Creates the child service process.
exec()	Executes the test service program.
getpid()	Obtains the process ID of a process.
getppid()	Obtains the parent process ID.
waitpid()	Waits for the service process to terminate.
kill()	Sends a signal to the service process.
SIGTERM	Requests safe termination of the service.
Process PID	Identifies and tracks the running service.
/proc/<PID>/status	Provides information about a Linux process.
VmSize	Shows the virtual memory size of the process.
VmRSS	Shows the resident memory used by the process.
File I/O / Logging	Records service execution information in a log file.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030380	Yamini	• Supervisor Core: Developed and tested the Linux Service Process Supervisor. • Implemented service creation using <code>fork()</code> and <code>exec()</code>. • Implemented service PID tracking. • Implemented Start Service functionality. • Implemented Stop Service functionality using <code>SIGTERM</code>. • Implemented Service Status functionality. • Implemented Restart Service functionality. • Implemented safe supervisor shutdown. • Tested the service lifecycle operations in Ubuntu/Linux. • Maintained the project repository and testing evidence.
2520030364	Lekhana	• Signals and Process Control: Worked on signal-based process control. • Studied the use of <code>SIGTERM</code> for service termination. • Assisted with process-control and service lifecycle testing.
2520030613	Anikha	• Monitoring, Logging and Documentation: Worked on process monitoring and <code>/proc</code> information. • Assisted with memory monitoring using <code>VmSize</code> and <code>VmRSS</code>. • Assisted with logging, testing and project documentation.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Ubuntu / Linux	Development and execution environment.
C	Programming language used for the project.
GCC	Compiles the C source programs.
Linux Terminal	Used to compile, execute and test the programs.
Linux System Calls	Used for process creation, execution and control.
POSIX Signals	Used for service termination and process control.
<code>/proc</code> Filesystem	Used to obtain process and memory information.
Git	Used for version control.
GitHub	Used to store and manage the project source code, reports and testing evidence.
Nano	Used to create and edit source and documentation files.

8. Expected Outcome

The project is expected to provide a functional Linux Service Process Supervisor capable of managing a test service process.

The expected outcomes are:

1. The supervisor successfully creates the service using `fork()`.
2. The service program is executed using `exec()`.
3. The supervisor displays the service PID after starting it.

4. The supervisor correctly identifies whether the service is RUNNING or STOPPED.
5. The service can be safely terminated using SIGTERM.
6. The service can be restarted and assigned a new PID.
7. Process information can be obtained through /proc/<PID>/status.
8. Memory information such as VmSize and VmRSS can be displayed.
9. Service execution information can be recorded in log files.
10. The supervisor can safely shut down after terminating any running service.

The implemented testing demonstrated these service lifecycle and process monitoring operations successfully.

9. GitHub Repository

GitHub Repository Link:

Repository Name: K LH-CSE-2026-27-Team19-LinuxServiceProcessSupervisor

Repository Created:

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name:

Signature:

Remarks:

Faculty Use

Project Approved:

☐ Yes ☐ No

Date:

Faculty Signature: